AgentPProf: Semantic Profiling for AI Agent Traces

Abstract

AI agents produce complex execution traces spanning prompts, LLM calls, tool invocations, GUI actions, and system effects. As deployments grow to hundreds of sessions, developers need aggregate views to identify which task categories consume the most budget, where failures concentrate, and which workflows waste effort. Existing observability tools organize events into fixed session/span hierarchies suited for single-run debugging but unable to fold events across sessions by semantic work category. The consequence is that failures, safety violations, and wasted effort remain unattributable to the task categories that cause them. The root cause is structural: agent prompts are natural language, so two prompts with the same intent share no identifiers that a profiler could merge. We show that agent profiling reduces to two abstractions—uniform *operations* and query-time *operation stacks*—once intent recognition restores the deterministic labels that flame-graph folding requires. AgentPProf implements this model as a Rust profiler with pluggable intent-recognition backends. On 325 real agent sessions, semantic labels separate over 90% of system-effect weight that flat views merge. On four public datasets with hidden ground-truth labels (34,539 operations), operation-stack profiling reduces inspection work to 9.4% versus flat summaries and group fragmentation by 45% versus fixed-session drilldown.

1 Introduction

AI agent systems produce heterogeneous execution traces spanning prompts, LLM calls, tool invocations, GUI actions, and system effects. As deployments scale to hundreds of sessions, developers need cross-session aggregate views to identify which task categories consume the most budget, where failures concentrate, and which workflows waste effort.

However, no existing tool provides cross-session resource attribution by semantic category. In a real development project, `cargo test` appears 2,903 times across 325 sessions, yet no trace tool can attribute these executions to the review, debugging, or refactoring prompts that triggered them.

The aggregation problem itself is not new—CPU profilers solved it for programs decades ago. Flame graphs [10] fold millions of function-call samples into a single chart where width represents cost. The key enabler is *deterministic* function names that merge identical call stacks. Agent traces break this precondition because prompts are natural language, and two prompts expressing the same intent share no characters (§2).

Existing agent observability tools do not close this gap. LangSmith [12], Langfuse [13], Phoenix [1], and OpenTelemetry GenAI [21] organize events into per-execution span trees, enabling single-run debugging but not cross-session aggregation. Datadog LLM Observability [5] clusters user messages by topic, and Laminar [11] extracts structured signals from traces. Both characterize *input distributions* ("30% of users ask code questions") but not *resource attribution* ("code-review tasks consume 40% of the token budget"), because clustering at the request level does not propagate labels to downstream system effects.

We observe that three interdependent elements restore cross-session aggregation for agent traces: a uniform *operation* record replaces the missing call-stack sample, a query-time *operation stack* (a field projection, not an execution artifact) replaces the call stack, and *intent recognition* restores the deterministic labels that folding requires. Remove any one and the model degrades: without labels, aggregation mixes intents. Without query-time stacks, the hierarchy is locked at ingestion.

This paper presents AgentPProf, a Rust profiler built on this model. On 325 real Codex/Claude sessions, semantic labels separate 90% of system-effect weight that flat views merge. On four public datasets with hidden ground-truth labels, operation-stack profiling reduces inspection work to 9.4% versus flat summaries and group fragmentation by 45% versus fixed-session drilldown.

This paper makes three contributions:

(1) **Insight: agent profiling reduces to two abstractions plus intent recognition.** We show that agent traces lack the inherent call stack that makes CPU flame graphs work. Uniform operation records, query-time stack projections, and intent-derived deterministic labels are jointly necessary to close this gap—any two without the third fall back to either flat GROUP BY or fixed-hierarchy debugging (§2–§3).
(2) **System: AgentPProf.** A profiler that realizes this model with pluggable intent recognition (regex rules, local LLM tagging, unsupervised clustering) and produces output (pprof, SVG, JSON) consumable by both developers and automated analysis agents (§4).
(3) **Evaluation via hidden-label localization.** We score profiler output against hidden ground-truth trajectory labels (failure, safety, redundancy, and human-task boundaries), analogous to injecting known bottlenecks in CPU profiler evaluation (§5).

2 Background and Motivation

2.1 Existing Agent Trace Models

Current agent observability tools organize traces around fixed hierarchies. OpenTelemetry represents an execution as a *trace* containing a tree of *spans*, each with a name, duration, and key-value attributes. LangSmith and Langfuse group spans into *sessions* and display them as timelines or waterfall charts. Phoenix and OpenInference add GenAI-specific span types for LLM calls, tool invocations, retrieval steps, and agent reasoning.

These models share a structural property: the session or span tree is the skeleton of the data. Every event belongs to exactly one span, and spans nest in a fixed parent-child hierarchy determined at execution time. This design is effective for debugging because a developer can locate a failing span, inspect its attributes, and trace its parent chain to understand the execution context.

2.2 Fixed Hierarchies and Profiling

Profiling requires a fundamentally different operation, *folding* events across executions by semantic category so that width represents

aggregate cost. Fixed session/span hierarchies obstruct this in three ways.

First, sessions are boundaries, not categories. A single session may contain review, debugging, and refactoring phases, and splitting by session assigns all of them to the same bucket. Conversely, two sessions performing the same task produce separate buckets that cannot be merged without external logic.

Second, span names describe mechanisms (`LLMCall`, `ToolInvoke`, `ChatCompletion`), not user intent (`review`, `debug`, `test`). Grouping by span name tells you how many LLM calls occurred, not which categories of work consumed the budget.

Third, these limitations compound in practice. A common system effect like running a test suite may appear thousands of times across sessions, but a flat process summary shows only a single counter with no way to attribute executions to the semantic intent that triggered them. We quantify this disaggregation in RQ1 (§5).

2.3 CPU Profiling and Agent Traces

CPU profilers solve the equivalent aggregation problem for programs. A flame graph [10] represents a profile as a set of weighted *samples*, each attached to a *call stack* (an ordered list of function frames). Identical stacks merge, and width represents aggregate cost. The key enabler is that function names are *deterministic*. The same code path always produces the same frames.

Agent traces break this precondition in two ways. First, prompts are natural language and lack deterministic identifiers. Second, there is no inherent call stack at all: agent events (prompts, tool calls, GUI actions, process effects) have no runtime-determined parent-child nesting analogous to function calls. Pprof's `tagroot`/`tagleaf` options add label-derived pseudo-frames to an existing execution stack, but agent traces have no execution stack to augment. The profiling structure must be constructed entirely from semantic fields at query time.

These observations yield three design requirements:

- **R1: Uniform record type.** Agent traces mix prompts, LLM calls, tool invocations, GUI actions, process events, and benchmark labels. The profiler needs a single record type that can represent all of them without type-specific objects.
- **R2: Query-time aggregation.** The same data must support multiple aggregation shapes (by task, phase, action, session, or dataset) without rebuilding the trace. Sessions and spans must be fields, not fixed hierarchy nodes. This subsumes diagnostic views: failure, safety, and quality labels are fields that participate in the same aggregation.
- **R3: Deterministic labels for folding.** Free-form prompts must be mapped to short, stable, repeatable labels before folding. This *intent recognition* step has no analogue in CPU profiling, where function names are already deterministic.

3 Design

The three requirements (a uniform record type, query-time aggregation, and deterministic labels for folding) drive the design. We introduce two abstractions that satisfy R1 and R2, operations (§3.1) and operation stacks (§3.2), and a pluggable intent-recognition mechanism (§3.3) that satisfies R3.

Table 1: Built-in views. Each shares the same operation set but selects different activities, stack structures, and weights.

View	φ (select)	w (width)	Question
`tokens`	LLM calls	token count	Budget distribution?
`time`	All timed ops	duration (s)	Time distribution?
`files`	File-effect ops	event count	Repo coverage?
`network`	Net-effect ops	event count	External contacts?

3.1 Operations (R1: Uniform Record Type)

A profiler should not distinguish between prompts, tool calls, and process events at the object-model level. AgentPProf uses a single abstraction. An `operation` is a weighted record with string fields and additive measures.

Each operation carries string fields (`project`, `agent`, `session`, `prompt_tag`, `kind`, `model`, `path`, `domain`, `status`, …) and additive measures (token count, duration, occurrence count). A prompt, an LLM call, a tool invocation, a file read, a GUI click, and a process event are all operation records that differ only in which fields are populated.

3.2 Operation Stacks (R2: Query-Time Aggregation)

An `operation stack` is an ordered list of fields chosen at query time. Folding operations by that list produces the standard folded-stack representation:

```
CPU:   main;parse;tokenize          42
Agent: prompt:debug;call:llm;tool:bash  4200
```

Unlike CPU profiling, where the call stack is determined by execution, the stack is determined by the query. The same operations can be folded as a flat task summary, a fixed-session tree, a semantic phase/action hierarchy, or a dataset-native trajectory. Changing the fields changes the aggregation without changing the data.

Query-time stack selection directly addresses the fixed-hierarchy problem from §2. Sessions and spans are optional operation fields, not structural skeleton nodes. Including `session` in the stack produces a per-session drilldown (equivalent to a trace tree). Excluding it produces a cross-session aggregate that existing trace tools do not natively support. The same data supports both debugging and profiling views.

More precisely, a *view* is a triple (φ, σ, w). The predicate φ selects which operations participate, the stack function σ maps each operation to a frame sequence, and the weight function w assigns a nonnegative measure. The view formalism supports arbitrary user-defined views. Four built-in defaults (Table 1) answer common questions.

3.3 Intent Recognition (R3: Deterministic Labels)

Operations and stacks handle heterogeneity and flexible aggregation, but folding still requires deterministic frame labels—the precondition that agent prompts violate (§2). AgentPProf solves this by mapping free-form prompts to short, stable labels before stack construction.

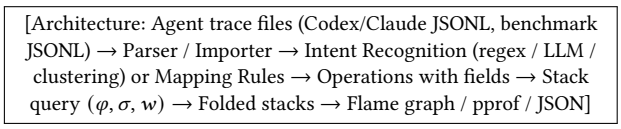

Figure 1: System architecture. Local agent histories enter through `agent-session` parsing. Public datasets enter as operation JSONL. Both paths produce operations. The stack query and output are identical.

The design uses a pluggable tagger framework with three backends. First, *regex rules* are the production default. Rules have the form `KIND:TAG=REGEX`, where `KIND` is `prompt`, `llm`, or `all`, `TAG` is a short lowercase label, and `REGEX` is a pattern. Rules are evaluated in order; the first match wins. The workflow is human–agent collaborative: the developer runs AgentPProf, observes the unmatched rate and sample prompts, adds rules, re-runs, and iterates until unmatched drops below 5%. This typically takes 5–10 rounds. The resulting rule set is deterministic, reproducible, and version-controllable. Second, a *local LLM tagger* handles exploration or complex multilingual prompts. A local model (e.g., a 3B-parameter GGUF via llama.cpp) generates a single-word label per prompt. Grammar-constrained decoding ensures output validity. Labels are cached, so re-runs with unchanged prompts incur zero LLM calls. Third, *unsupervised clustering* via TF-IDF vectorization and K-Means discovers natural prompt groups without predefined rules, feeding rule authoring.

The one-word label design is deliberate. Flame-graph frames must be short because long summaries make charts unreadable. Labels serve navigation and aggregation, not correctness verdicts.

For structured traces such as public benchmark datasets, where action types, tool names, and quality annotations are explicit fields, intent recognition is unnecessary. AgentPProf supports deterministic *mapping rules* that derive new operation fields from existing ones before stack construction. A *profile spec* records the complete analysis configuration (data sources, mappings, predicates, stack fields, and outputs) for deterministic replay. This dual path explains why AgentPProf handles both real agent sessions (which need labeling) and public benchmark traces (which need mapping rules). Both paths produce operations with fields, and the stack query works identically on both.

4 Implementation

AgentPProf is an offline, post-hoc profiler implemented as a Rust CLI (Figure 1). It reads recorded traces, not live events. The `agent-session` crate reads Codex and Claude Code local JSONL history files and reconstructs prompts, LLM calls, tool invocations, and their triggered file and network effects. Each prompt opens a span. LLM calls, tool calls, and system effects within that span inherit its semantic label, connecting intent to system behavior without runtime instrumentation. Public benchmark datasets enter as operation JSONL with mapping rules deriving fields before stack construction. Standard Chrome Trace Event format can also be imported, with non-AgentPProf protocol fields preserved as operation fields.

Output formats include pprof protobuf (compatible with `go tool pprof -http`), folded-stack text (compatible with `flamegraph.pl`), standalone SVG flame graphs, and JSON summaries. All outputs are deterministic given the same input and profile spec.

AgentPProf is the offline profiling component of the broader AgentSight observability framework. AgentSight provides real-time eBPF-based capture of SSL traffic, process events, and file/network effects. AgentPProf aggregates the recorded traces into semantic profiles. The two are complementary. AgentSight answers "what happened during this run" (debugging), AgentPProf answers "where did the budget go across runs" (profiling).

5 Evaluation

We use two classes of data. *Real agent sessions*: 325 readable local sessions (198 Codex, 50 Claude, 77 Claude subagent) from a multi-month development project, containing 183,714 system observations. *Public labeled traces*: 15 families covering web, mobile, desktop, software, API, and dialogue agents (Table 2), totaling 47,590 operations through mapping rules. All public traces are real agent or human executions collected and labeled by their respective benchmark projects. Four oracle-rich families (AgentRewardBench, SATraj-OS, AgentNet, OSWorld-Human) provide hidden ground-truth labels for RQ2. We measure semantic separation (mixed-weight percentage), localization quality (AP, recall, work-to-first-positive), and label agreement (V-measure, boundary F1).

5.1 RQ1: Does Semantic Profiling Separate What Flat Views Mix?

The strongest argument against semantic profiling (cross-session aggregation by intent-derived labels) is that traditional tools "can already see the same information." RQ1 tests this claim directly by measuring how much system-effect weight falls into *mixed-intent* buckets (those containing observations from multiple semantic regions) under different projection choices.

Table 3 shows the result on 325 real Codex/Claude sessions. Without semantic labels, 90.4% of system-effect weight falls into mixed buckets. A flat `cargo test` counter shows 2,903 executions but cannot distinguish whether they came from `review`, `debug`, `refactor`, or `test` prompts. Adding only the prompt label reduces mixed weight to 36.7% and residual to 7.5%. The session label alone barely helps (84.4%), because sessions typically contain multiple intent phases.

A concrete example illustrates the scale. `process:cargo; effect:test; status:ok` appears under 48 distinct prompt labels. The top label (`refactor`) accounts for only 32.0%. The remaining 68.0% comes from `review`, `research`, `test`, `explain`, and others. A flat summary or process audit merges all of these.

As a robustness check, a label-permutation test (1,000 shuffles within each session) confirms the separation is not random ($p = 0.001$). This result assumes label quality, a dependency we validate in RQ3 for structured traces (V-measure above 0.7 on 7/9 datasets). For the regex-derived labels on real sessions, the permutation test confirms non-random separation, but direct quality assessment against human-annotated ground truth remains future work.

Table 2: Public labeled trace families. All traces are real agent or human executions. The bottom four rows provide oracle labels for RQ2.

Family	Native labels	Access	Trace origin	Eval. role
WebLINX [18], Mind2Web [6], Agent-Trek [26], WebShop [28]	Web actions, tasks, rewards	HF / public	Real web agents	Coverage
API-Bank [14], Tool-Bench [22], tau-bench [29]	API calls, outcomes	HF / JSONL	Real API agents	Coverage
SWE-agent [27]	Issue trajectories	HF viewer	Real coding agent	Coverage
AndroidCtrl [15], GUI-Odyssey [17], Scale-CUA [16]	Mobile/GUI actions	Public/HF	Real mobile agents	Coverage
AgentRewardBench [19]	Success, side-effect, looping	HF annotations	Real BrowserGym agents	RQ2 tasks
SATraj-OS [4]	Safety, reward, attack type	HF config	Real desktop agents	RQ2 task
OSWorld-Human [24]	Human grouped-action traces	GitHub JSON	Real human OS use	RQ2 task
AgentNet [25]	Step correctness, redundancy	HF JSONL	Real desktop agents	RQ2 tasks

Table 3: Semantic-axis ablation on 183,714 system observations from 325 real sessions. All rows preserve the same total weight.

Projection	Unique stacks	Mixed wt.%	Residual%
No semantic labels	11,967	90.4	44.7
Session label only	15,027	84.4	33.4
Prompt label only	24,703	36.7	7.5
Session + prompt	26,829	0.0	0.0

RQ1 also tests whether query-time stack selection (R2) adds value beyond flat label grouping. The same two-abstraction model covers all 15 public trace families (Table 2) without type-specific objects, validating R1. On the four oracle-rich families, the same 13,265 operations fold from 9 dataset stacks to 57 phase stacks, 226 semantic stacks, 455 action stacks, or 3,757 fixed-session stacks by changing only the stack fields. A flat GROUP BY on a single label cannot produce these views because different stack depths answer different questions (task-level budget vs. action-level duplication vs. session-level drilldown), and the profiler produces all of them from the same operations.

The multi-view design is further validated by metric divergence. The `time` and `tokens` views of the same sessions share only 7/10 top prompt categories (Spearman $\rho = 0.623$). `network`-tagged prompts rank 8th by time (I/O wait) but 93rd by tokens.

These quantitative results translate to concrete aggregate insights. The `tokens` view (Figure 2) reveals that `review`-tagged prompts dominate the model budget, followed by `git`, `code`, `edit`, and debug. A timeline of 80,000 individual LLM calls cannot produce this distribution without external aggregation. The `files` view shows debug-tagged prompts across different sessions repeatedly accessing the same source files and identifies accesses outside the project root, supporting security auditing. The `network` view attributes domain contacts to prompt categories.

[Flame-graph screenshots: tokens view (top) and files view (bottom) showing budget distribution and file-access patterns across sessions]

Figure 2: Real flame graphs from a development project. Top: `tokens` view (width = token count). Bottom: `files` view (width = file-operation count).

5.2 RQ2: Can Profiler Groups Localize Hidden Labeled Problems?

RQ2 asks whether the profiler's top groups correspond to real labeled problems, specifically whether categories that concentrate failures, safety violations, or wasted effort surface at the top of ranked output. The methodology is analogous to injecting known bottlenecks in a CPU profiler evaluation. Public benchmark datasets provide ground-truth labels (failure, safety violation, redundant step, human-task boundary); we hide these labels, run the profiler, rank the resulting groups, and check whether the top groups contain the hidden positives.

The benchmark uses six tasks across four datasets (AgentReward-Bench looping and side-effect, SATraj-OS unsafe operation, Agent-Net incorrect and redundant step, and OSWorld-Human group start), totaling 34,539 operations and 3,699 positives. Six views (flat, fixed-session, dataset-native, raw-action, operation-stack, label-drilldown) and four rankers (width, visible-risk, query-aware, oracle).

Table 4 summarizes the results. Flat summaries recover all positives but require inspecting everything (Work@5 = 1.0). Fixed-session drilldown finds a first positive quickly (WTFP = 0.004) but fragments the workload into 285 groups with only 2.3% top-five recall. Operation-stack query-aware profiling inspects 9.4% of the work in the top five groups, achieves 39.0% budget-30 recall, and reduces groups to 157.5.

The result is a Pareto tradeoff, not dominance. The comparison also serves as evidence for the two-abstraction model (§3). Fixed-session drilldown and dataset-native hierarchy are alternative abstractions that fix the hierarchy at session or dataset boundaries. Operation stacks subsume both as special cases (include `session`

Table 4: RQ2 hidden-label localization (medians across six tasks). Hidden policies serve only as upper bounds. WTFP = work-to-first-positive. "—" = group count varies by task; no single median is reported.

Policy	Hidden	AP	R@5	F1@5	Work@5	R@30%	WTFP	Groups
flat:width	0	0.168	1.000	0.287	1.000	0.000	1.000	1.0
fixed-session:query-aware	0	0.348	0.023	0.043	0.016	0.356	0.004	285.0
dataset-native:query-aware	0	0.357	0.867	0.282	0.854	0.338	0.058	—
raw-action:query-aware	0	0.274	0.244	0.220	0.151	0.333	0.037	—
op-stack:query-aware	0	0.312	0.188	0.198	0.094	0.390	0.038	157.5
op-stack:oracle-upper	1	0.599	0.300	0.403	0.044	0.564	0.012	—
label-drilldown:oracle	1	1.000	0.670	0.797	0.115	1.000	0.038	—

in the stack to recover fixed-session; include `dataset` to recover dataset-native). On localization, operation stacks improve top-five recall over fixed-session on 5/6 tasks, reduce median group count by 45%, and save 90% of inspection work versus flat summaries. Fixed-session drilldown retains lower work-to-first-positive on 4/6 tasks, confirming that profilers and debuggers are complementary.

Statistical validation supports these comparisons. Bootstrap (10,000 resamples) and label-permutation (2,000 trials) confirm stable AP deltas exceeding the 95% prevalence/group-size null on all six tasks.

Beyond localization, we test whether profiler output suggests concrete configuration improvements. Executable profile-spec patches modify stack fields, mapping rules, or rankers, then re-run over the same visible operations. Five of six patches improve AP, top-five lift, and first-positive work (median AP delta 0.038, median top-five lift delta 0.575). Three reusable patterns emerge: (1) safety/side-effect tasks benefit from field and ranker changes; (2) step-quality tasks benefit from stack-based localization followed by fixed-session drilldown for examples; (3) human-boundary tasks require boundary-derived fields. The rejected OSWorld-Human patch shows that visible rank features alone are insufficient for boundary objectives. Folding held-out boundary-derived fields improves AP from 0.240 to 0.258 and reduces groups from 108 to 74, with an explicit work tradeoff.

Rank-feature ablations identify 7 critical feature rows (`repeat_signal`, write-action, `status=success`) and 3 misleading rows. Guided repairs improve AP on 2/3 misleading cases but use offline scored feedback, not an automatic learned ranker.

5.3 RQ3: How Reliable Are the Derived Labels?

RQ1 and RQ2 assume that semantic labels are useful. RQ3 tests the label-derivation pipeline itself. For structured traces (public datasets), we evaluate the *mapping* path against ground-truth annotations. For free-form traces (real sessions), we evaluate the *intent-recognition* path at the scale and syntax level. Quality assessment of the LLM/regex tagger against free-form ground truth remains future work.

On 325 real sessions, the LLM tagger processes 118,021 tag requests (2,859 prompt rows, 114,837 LLM-call events), of which 82,886 hit the label cache and 35,136 require real llama.cpp calls to a local 3B-parameter model. All requests complete without tag failures. On a fixed set of 300 redacted fragments, three model sizes (0.6B, 1.1B, 3B) all produce 900/900 syntactically valid outputs with p95 latencies of 23, 18, and 31 ms respectively.

Table 5: Leave-dataset-out mapping quality. V-measure between mapping-derived `phase` and native `action` labels. 13,265 operations from the four oracle-rich families across 9 held-out evaluations.

Held-out dataset	Ops	V-measure	Boundary F1
mind2web	774	1.000	1.000
webshop-expert	2,504	1.000	1.000
swe-agent	496	0.926	0.962
weblinx-chat	500	0.872	0.860
agenttrek	200	0.862	—
gui-odyssey	7,868	0.811	0.842
android-control	9	0.716	0.727
toolbench	866	0.134	0.353
api-bank	48	0.000	—

Public benchmark datasets provide native action labels that serve as ground truth for mapping quality. We run a leave-dataset-out evaluation. For each of 9 datasets, we train mapping rules on the other 8 and measure V-measure between the mapping-derived `phase` field and the dataset's native `action` labels, plus boundary F1 between mapped phase boundaries and native action boundaries.

Table 5 shows the results. On 7/9 datasets, V-measure exceeds 0.7, indicating strong agreement between mapping-derived and native labels. The two low-scoring datasets (toolbench, api-bank) have atypical action vocabularies that the held-out mapping rules do not cover. Mapping reduces unique stacks on 6/9 datasets with zero negative regressions.

A supervised boundary-derivation backend further validates the field pipeline. On OSWorld-Human held-out adjacent pairs, a learned boundary predictor achieves F1 0.77, beating the best simple baseline (`always_boundary`, F1 0.71). Learned backends beat simple baselines on 4/5 tested boundary families, while AgentRewardBench looping is fully explained by `repeat_signal_change` (baseline F1 1.0). These baseline comparisons support field derivation as a guarded configuration step, not automatic intent discovery.

Two mapping-rule variants (differing in API/tool phase precedence) run on the same 13,265 operations and produce identical V-measure on 7/9 datasets, with the two divergent datasets differing by $\leq$0.17. Mapping quality is robust to minor rule variations on well-covered action vocabularies. Direct cross-backend comparison between the regex and LLM taggers on the same free-form prompts remains future work.

Syntactic validity does not imply semantic quality. TinyLlama 1.1B achieves high exact-stability (279/300 identical across repeats) but concentrates on a narrow label set. The 3B model produces a broader vocabulary (328 unique prompt labels) but includes noisy entries. A deterministic aliasing step reduces unique labels from 1,546 to 1,364, raising top-20 support coverage from 93.7% to 95.2% without overwriting raw labels. This pipeline manages noise but does not establish semantic adequacy. The ground-truth and cross-backend checks above provide that.

All results are backed by profile specs for deterministic replay. We re-execute 76 tracked specs twice (152 invocations), and all produce identical output with median runtime 1.6 s per spec.

6 Related Work

Flame graphs [10] and pprof [9] establish the folded-stack lineage. Profiles are weighted samples with location hierarchies. Pprof supports sample labels and can visualize them as pseudo stack frames through tag-root/tag-leaf options. Perfetto generalizes trace ingestion with SQL-based analysis and derived events [8]. These systems support query-time aggregation, so AgentPPProf does not claim novelty for flexible aggregation alone. The difference is structural. Pprof's tagroot adds label-derived frames to an existing execution call stack. Agent traces have no execution stack, so the entire profiling structure must be constructed from semantic fields. The evaluation also differs: AgentPPProf scores profiler output against hidden agent-trajectory labels, a methodology with no analogue in CPU profiling.

OpenTelemetry GenAI and OpenInference standardize GenAI spans [2, 21]. LangSmith [12], Langfuse [13], and Phoenix [1] provide trace visualization, session management, evaluations, and dashboards. AgentOps [7] argues for structured agent observability covering goals, plans, tools, and artifacts. These systems excel at single-execution debugging. AgentPPProf complements them with cross-session aggregate profiling. Datadog LLM Observability [5] and Laminar Signals [11] take steps toward semantic aggregation by clustering user messages or extracting structured events, but they characterize *input distributions*, not *resource attribution*.

AgentRx [3] releases annotated failure trajectories and uses constraint checking plus an LLM judge for root-cause localization. TELBench [23] builds span-localization benchmarks from semantic error annotations. AgentAtlas [20] argues that outcome leaderboards miss trajectory-level quality. These systems perform *span-level* fault localization within a single execution: which step failed and why. AgentPPProf performs *category-level* aggregate profiling, identifying which categories of work produce the most failures, consume the most budget, or generate repeated system effects across sessions.

7 Conclusion

Agent observability needs profiling, not only debugging. The root cause is structural: natural-language prompts lack the deterministic identifiers that make flame-graph folding work. AgentPPProf shows that two abstractions, operations and operation stacks, close this gap once intent recognition restores deterministic labels. Scoring profiler output against hidden trajectory labels confirms that semantic profiling separates information that flat views merge and localizes labeled problems with less work than both flat summaries and fixed-session drilldown. Mapping-derived labels agree with ground-truth annotations on 7/9 held-out public datasets.

Profilers and debuggers are complementary. Fixed-session drilldown finds a first positive faster on 4/6 tasks (RQ2), while the profiler identifies which categories deserve attention. AgentPPProf supports both by including or excluding session fields in the stack query. The profiler's structured output (JSON, pprof, folded stacks) is designed for automated consumption by analysis agents. Evaluation of agent-driven diagnosis is future work.

The main limitation is single-project real-session data. Multi-project evaluation and trace-ecosystem integration (importing OpenTelemetry and LangSmith traces as operations) are the most impactful next steps.

References

[1] Arize AI. 2024. Arize Phoenix. https://arize.com/docs/phoenix.

[2] Arize AI. 2024. OpenInference Specification. https://arize-ai.github.io/openinference/spec/.

[3] Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475. doi:10.48550/arXiv.2602.02475

[4] Xinquan Chen et al. 2026. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. arXiv preprint arXiv:2605.06230. doi:10.48550/arXiv.2605.06230

[5] Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.

[6] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

[7] Liming Dong, Qinghua Lu, and Liming Zhu. 2024. AgentOps: Enabling Observability of LLM Agents. arXiv preprint arXiv:2411.05285. doi:10.48550/arXiv.2411.05285

[8] Google. 2024. Perfetto Trace Processor. https://perfetto.dev/docs/analysis/trace-processor.

[9] Google. 2024. pprof. https://github.com/google/pprof.

[10] Brendan Gregg. 2011. Flame Graphs. https://www.brendangregg.com/flamegraphs.html.

[11] Laminar. 2025. Signals: Structured Event Extraction from Traces. https://docs.lmnr.ai/signals/introduction.

[12] LangChain. 2024. LangSmith Observability. https://docs.langchain.com/langsmith/observability.

[13] Langfuse. 2024. Langfuse Observability. https://langfuse.com/docs/observability/overview.

[14] Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 3102–3116.

[15] Wei Li, William E. Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. 2024. On the Effects of Data Scale on UI Control Agents. arXiv preprint arXiv:2406.03679. doi:10.48550/arXiv.2406.03679

[16] Zhaoyang Liu et al. 2025. ScaleCUA: Scaling Open-Source Computer Use Agents with Cross-Platform Data. arXiv preprint arXiv:2509.15221. doi:10.48550/arXiv.2509.15221

[17] Quanfeng Lu, Wenqi Shao, Zitao Liu, Fanqing Meng, Boxuan Li, Botian Chen, Siyuan Huang, Kaipeng Zhang, Yu Qiao, and Ping Luo. 2025. GUI Odyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

[18] Xing Han Lù, Zdeněk Kasner, and Siva Reddy. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. arXiv preprint arXiv:2402.05930. doi:10.48550/arXiv.2402.05930

[19] Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J. Pal, and Siva Reddy. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. arXiv preprint arXiv:2504.08942. doi:10.48550/arXiv.2504.08942

[20] Parsa Mazaheri, Yifei Shen, and Wenpeng Zhong. 2026. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. arXiv preprint arXiv:2605.20530. doi:10.48550/arXiv.2605.20530

[21] OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. https://github.com/open-telemetry/semantic-conventions-genai.

[22] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerber, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

[23] Jiaming Wang, Ziteng Feng, Jiangtao Wu, Ruihao Li, Qianqian Xie, Yuxiang Ren, He Zhu, Xueming Han, Fanyu Meng, Junlan Feng, and Jiaheng Liu. 2026. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. arXiv preprint arXiv:2606.02060. doi:10.48550/arXiv.2606.02060

[24] WukLab. 2025. OSWorld-Human. https://github.com/WukLab/osworld-human.

[25] xlangai. 2025. AgentNet. https://huggingface.co/datasets/xlangai/AgentNet.

[26] Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. 2024. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. arXiv preprint arXiv:2412.09605. doi:10.48550/arXiv.2412.09605

[27] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

[28] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.

[29] Shunyu Yao, Karthik Narasimhan, and Juntao Cao. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv preprint arXiv:2406.12045. doi:10.48550/arXiv.2406.12045